Nanochip

An AI-native architecture sandbox

PROPOSED SYSTEM

Install and open one student workspace

Target: pip install nanochip · notebooks / CLI · inspect data · edit components · compare runs

Workload and experiment contract

Kimi-derived workload · objectives · required checks
Permitted changes · constraints · experiment budget

Learning components

Data and representations
Inspect · cover · group · split
States, actions, and tool results

Train or fine-tune
Small local model
Actual parameter updates

Versioned checkpoint
Weights · tokenizer · lineage

Context / retrieval
Specifications · allowed evidence

Design experiment

Decision methods
Predict · propose · search
Local or optional external model
Conventional alternatives

action

Experiment harness
Build candidate · invoke tools
Enforce checks and budgets
Preserve identities and retries

execution status
and resource use

Feedback selects the next action
Checkpoint fixed within each campaign

Observations and records
Metrics · failures · artifacts · cost
Raw logs + qualified observations

Curate development records for later training or adaptation

Model parameters change between campaigns.

One Nanochip target

Nanochip candidate
Software · schedule · RTL
One versioned design

Functional evaluation
Numerical and protocol checks
Simulation · cycles · traffic

selected candidates

Physical implementation
Synthesis · place and route
Logic + realized memories

Layout and qualification
GDSII · timing · DRC / LVS
Equivalence · electrical · antenna

Pinned simulator and physical tools
Candidate identity retained throughout

Campaign evidence

Independent evaluation
Fresh runs through the same harness
Baseline · transfer · total cost

Held-out cases
Unseen during development
Fixed evaluation protocol

Architect evaluates and decides
Accept · revise · reject · retain baseline
Architecture outcome and the cost of reaching it